This chapter introduces the fundamental concepts of program analysis, exploring how we can mathematically assign meaning to syntax and systematically reason about program behaviors.

---

1. Overview and Motivation

Program analysis is the systematic examination of a program to determine its properties. We apply it for various reasons:

- Finding bugs (historical fun fact: the first "bug" was an actual moth found in the Harvard Mark II in 1947).
- Optimizing performance and building compilers.
- Detecting security vulnerabilities and improving software maintainability.

To check if a program behaves as intended, we must verify if the code satisfies a specific semantic property. However, this is fundamentally constrained by **Rice's Theorem**: there exists no algorithm that can automatically, universally, and exactly determine if a program satisfies a non-trivial semantic property.

Because we cannot have a tool that is simultaneously automatic, universal, and exact, we must rely on different techniques that trade off these traits:

- **Testing:** Unsound (can accept incorrect programs), but complete (does not reject safe programs).
- **Machine-Assisted Proving (e.g., Coq, Isabelle):** Sound and quasi-complete, but requires manual effort to supply invariants.
- **Finite-State Model Checking:** Automatic, sound, and complete, but only with respect to finite models (can fail to terminate on infinite states).
- **Conservative Static Analysis:** Automatic and sound, but incomplete (relies on over-approximation).

---

2. Formal Semantics and State

Semantics assigns mathematical meaning to syntax. We distinguish between the syntactic representation of a program (how it is written, c) and its computed function ($[[c]]$).

- **Memory State:** A state σ is a function mapping variables to integers: $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space:** The set of all possible states is denoted as $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ denotes a state identical to σ, except the variable $\mathtt{x}$ evaluates to n.

2.1 Forward Semantics

For deterministic programs, the denotational semantics models a program as a function from an input state to an output state: $[[c]] : \Sigma \to \Sigma_\perp$.

- The set $\Sigma_\perp = \Sigma \uplus \{\perp\}$ includes a special symbol $\perp$ to represent non-terminating executions.

2.2 Collecting Semantics

To consider all possible execution trajectories simultaneously (especially useful for non-deterministic programs or abstraction), we lift the semantics to operate on sets of states (the powerset $\wp(\Sigma)$).

- **Collecting Semantics:** $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.
- Defined as: $[[c]]P = \bigcup_{\sigma \in P} [[c]]\sigma$.

---

3. Approximations

Since exact analysis is impossible due to undecidability, we use approximations.

- **Over-approximations:** Expand the set of possible behaviors. If the over-approximated program doesn't reach a bad state, the exact program won't either.
  - *Pro:* Good for proving correctness (Sound).
  - *Con:* Can report bugs that don't exist (False Positives).
- **Under-approximations:** Restrict the set of possible behaviors. If the under-approximated program reaches a bad state, the exact program definitely will.
  - *Pro:* Good for bug-finding.
  - *Con:* Can miss actual bugs (False Negatives).

---

4. Denotational Semantics of IMP

We construct the meaning of a program compositionally.

4.1 Expressions

- **Arithmetic Expressions (Aexp):** $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$. Example: $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$.
- **Boolean Expressions (Bexp):** In collecting semantics, a boolean expression filters a set of states, keeping only those where the condition holds: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$. Example: $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$.

4.2 Commands and Control Flow

Command semantics $[[\cdot]] : \text{Com} \to \wp(\Sigma) \to \wp(\Sigma)$ are defined recursively:

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$.
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$.
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$.
- **Choice (Non-determinism):** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$.

4.3 Loops and Fixpoints

Loops are handled using the **Kleene Star** operator, which applies a command zero or more times: $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty}[[c]]^k P$. To encode a `while` loop, we apply the loop body c as long as condition b holds, and finally filter by $\neg b$:

$$[[\mathtt{while\ b\ do\ c}]]P \triangleq [[\neg b]] \bigcup_{k=0}^{\infty} ([[c]] \circ [[b]])^k P$$

This represents finding the Least Fixed Point of the semantic function mapping states through iterations of the loop.

---

5. Correctness and Preconditions

We use semantic definitions to formally prove if a program starting in a set of input states P will only result in valid output states Q.

- **Partial Correctness:** Denoted as $[[c]]P \subseteq Q$. This means that starting from P, the execution of c either does not terminate, or terminates in a state satisfying Q.

To systematically calculate correctness, we compute preconditions:

5.1 Dijkstra's Weakest Liberal Precondition (wlp)

Calculates the largest set of input states from which execution *either* diverges *or* ends in Q:

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

Thus, $[[c]]P \subseteq Q$ if and only if $P \subseteq wlp(c, Q)$.

5.2 Backward Semantics and Weakest Possible Precondition (wpp)

Backward semantics reverses the relation: $[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$. Hoare's Weakest Possible Precondition calculates the largest set of input states that have *at least one successful computation* reaching Q:

$$wpp(c, Q) = \{\sigma \mid [[c]]\sigma \cap Q \neq \emptyset\}$$

Conveniently, this maps directly to the backward semantics: $wpp(c, Q) = [[c]]_{op}Q$.

02. Denotational Semantics

This chapter formalizes how to assign mathematical meaning to programs, mapping syntax to its computed function. It focuses on collecting semantics, which is the foundation for defining approximations in static analysis.

---

1. Preliminaries and State

Semantics assigns meaning to syntax. Given a program written in a specific syntax c, its meaning is denoted as $[[c]]$.

- **State (σ):** A concrete memory state is a function mapping variables to integers, $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space (Σ):** The set of all possible states is $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ indicates a state identical to σ, except the variable $\mathtt{x}$ has the value n.
- **Concrete Domain:** In program analysis, we often work with sets of states. The concrete domain is the powerset of Σ, denoted as $\wp(\Sigma) \triangleq \{P \mid P \subseteq \Sigma\}$.

---

2. Semantics of Expressions

Expressions are evaluated relative to a given state or set of states.

2.1 Arithmetic Expressions (`Aexp`)

The denotational semantics of an arithmetic expression evaluates to an integer given a specific state: $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$.

- $[[n]]\sigma \triangleq n$
- $[[\mathtt{x}]]\sigma \triangleq \sigma(\mathtt{x})$
- $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$

2.2 Boolean Expressions (`Bexp`)

In collecting semantics, a boolean expression is treated as a filter over a set of states. It returns the subset of states that satisfy the condition: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$.

- $[[\text{true}]]P \triangleq P$
- $[[\text{false}]]P \triangleq \emptyset$
- $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$
- $[[\text{not } b]]P \triangleq P \setminus ([[b]]P)$

Note: $[[b]]P \subseteq P$ always holds.

---

3. Regular Commands and Collecting Semantics

For deterministic programs, semantics can be defined as $[[c]] : \Sigma \to \Sigma_\perp$, where $\Sigma_\perp = \Sigma \uplus \{\perp\}$ handles non-termination. However, to handle non-determinism and lay the groundwork for abstraction, we use **Collecting Semantics**, which maps a set of input states to a set of output states: $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.

By definition, $[[c]]P = \bigcup_{\sigma \in P} [[c]]\{\sigma\}$.

3.1 Base Commands

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$
- **Guard/Filter:** $[[b?]]P \triangleq [[b]]P$
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$
- **Non-deterministic Choice:** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$

3.2 Loops and Kleene Star

To execute a command an arbitrary number of times, we use the Kleene star operator ($c^\star$).

- $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty} [[c]]^k P$

This is mathematically resolved using Kleene's fixpoint theorem. The semantics of $[[c^\star]]P$ represents the smallest set of states that contains the initial states P and is closed under the execution of c: $[[c^\star]]P = P \cup [[c]]([[c^\star]]P)$

3.3 Encoding High-Level Control Flow

Standard control flow constructs are encoded using regular commands:

- **If-Then-Else:** $\mathtt{if\ b\ then\ c_1\ else\ c_2} \triangleq (b?; c_1) + (\neg b?; c_2)$
  - Semantics: $[[\mathtt{if\ b\ then\ c_1\ else\ c_2}]]P \triangleq [[c_1]]([[b]]P) \cup [[c_2]]([[\neg b]]P)$

- **While Loop:** `while b do c` $\triangleq (b?; c)^\star; \neg b?$
  - Semantics: $[[\texttt{while b do c}]]P \triangleq [[\neg b]] \left(\bigcup_{k=0}^{\infty}([[c]] \circ [[b]])^k P\right)$

---

4. Partial Correctness and Preconditions

We formalize correctness by checking if the reachable states conform to a specification.

Partial Correctness: Given a set of input states P and admissible outputs Q, a program c is partially correct if $[[c]]P \subseteq Q$. This means execution either diverges (does not terminate) or reaches a state in Q.

4.1 Dijkstra's Weakest Liberal Precondition (wlp)

The Weakest Liberal Precondition calculates the largest set of input states from which all terminating computations are guaranteed to end in Q.

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

Theorem: $[[c]]P \subseteq Q \iff P \subseteq wlp(c, Q)$.

4.2 Relational and Backward Semantics

Collecting semantics can be equivalently expressed as a relation $[[c]] \subseteq \Sigma \times \Sigma$, where $(\sigma, \delta) \in [[c]]$ means that starting from σ, state δ is reachable.

Backward Semantics: The opposite relation maps output states back to their possible input states.

$$[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$$

For example, $[[c_1; c_2]]_{op} \triangleq [[c_1]]_{op} \circ [[c_2]]_{op}$.

4.3 Hoare's Weakest Possible Precondition (wpp)

The Weakest Possible Precondition identifies the largest set of input states that have *at least one* successful computation ending in Q.

$$wpp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \cap Q \neq \emptyset\}$$

This directly corresponds to applying the backward semantics to the set of target states:

$wpp(c, Q) = [[c]]_{op} Q$.

03. Total Correctness

This chapter extends the reasoning of Hoare Logic to ensure **termination**. While partial correctness only guarantees the result *if* the program ends, **Total Correctness** ensures that the program *must* end and satisfy the postcondition.

---

1. Defining Total Correctness

A total correctness assertion is written using square brackets to distinguish it from partial correctness:

$$[P]\ c\ [Q]$$

1.1 Formal Definition

The triple $[P]\ c\ [Q]$ is valid (written $\models [P]\ c\ [Q]$) if and only if for every initial state σ such that $\sigma \models P$:

1. The execution of command c starting from σ **must terminate**.
2. The final state σ' **must satisfy** the postcondition Q ($\sigma' \models Q$).

1.2 Relation to Partial Correctness

Total correctness is the conjunction of partial correctness and termination:

$$[P]\ c\ [Q] \iff \{P\}\ c\ \{Q\} \wedge (\text{termination of } c \text{ from } P)$$

Consequently, proving total correctness is strictly harder than proving partial correctness.

---

2. Mathematical Foundations: Well-Founded Relations

To prove termination, especially for loops, we need a way to show that a program cannot run forever. We do this by mapping program states to a set that cannot have infinite decreasing sequences.

2.1 Well-Founded Relations

Definition: A binary relation $\prec$ on a set A is **well-founded** if there are no infinite descending chains $a_0 \succ a_1 \succ a_2 \succ \ldots$ in A.

- **Example (Well-founded):** The set of natural numbers with the standard "less than" relation $(\mathbb{N}, <)$. Any sequence of decreasing natural numbers must eventually hit zero.
- **Example (Not well-founded):** The set of integers $(\mathbb{Z}, <)$. One can decrease indefinitely into negative numbers $(0 > -1 > -2 \ldots)$.

2.2 Minimal Element Principle

A relation $\prec$ is well-founded if and only if every non-empty subset $Q \subseteq A$ has at least one **minimal element** m (an element such that no $x \in Q$ satisfies $x \prec m$).

---

3. Inference Rules for Total Correctness

Most rules for total correctness (Skip, Assignment, Sequence, Conditional) are identical to those in partial correctness because these constructs naturally terminate if their components do. The primary change is the **While Rule**.

3.1 The Total While Rule

To prove $[P]$ `while` b `do` c $[P \wedge \neg b]$, we introduce a **Variant** (also called a ranking function) t.

$$\frac{\forall z \in A.\ [P \wedge b \wedge t = z]\ c\ [P \wedge t \prec z]}{\vdash [P]\ \texttt{while}\ b\ \texttt{do}\ c\ [P \wedge \neg b]}$$

Requirements for the Rule:

- **Invariant:** P must be a valid loop invariant.
- **Well-founded Set:** $(A, \prec)$ must be a well-founded relation.
- **Variant Decrease:** In each iteration, the value of the expression t must strictly decrease according to $\prec$.

Because t decreases in a well-founded set, it cannot decrease forever; therefore, the loop must eventually terminate.

---

4. Practical Example: Counting Down

Consider a simple loop that decrements a counter:

$$\texttt{while}\ (x > 0)\ \texttt{do}\ x := x - 1$$

1. **Invariant (P):** $x \geq 0$.
2. **Variant (t):** The value of x itself.
3. **Well-founded Relation:** $(\mathbb{N}, <)$.
4. **Proof Step:** We must show that if the loop executes $(x > 0)$, the new value x' satisfies $x' < x$. Since $x' = x - 1$, this holds for all $x > 0$.

---

5. Weakest Preconditions (wp)

In total correctness, Dijkstra's **Weakest Precondition** (wp) specifically requires termination, unlike the Weakest Liberal Precondition (wlp).

- **Partial Correctness (wlp):** $wlp(c, Q) = \{\sigma \mid [[c]]\sigma \subseteq Q\}$.
- **Total Correctness (wp):** $wp(c, Q) = \{\sigma \mid [[c]]\sigma \neq \emptyset \wedge [[c]]\sigma \subseteq Q\}$.

The condition $[[c]]\sigma \neq \emptyset$ explicitly forces the existence of a final state, thereby ensuring termination.

04. Hoare Logic

This chapter introduces **Axiomatic Semantics** via **Hoare Logic**, a formal system used to reason about the correctness of computer programs by using logical triples.

---

1. Hoare Triples

The fundamental unit of reasoning in Hoare Logic is the **Hoare Triple**, which relates a program's behavior to logical assertions about the state before and after its execution.

A triple is written as:

$$\{P\}\ c\ \{Q\}$$

- P **(Precondition):** An assertion that must hold before the execution of command c.
- c **(Command):** The program or fragment being analyzed.
- Q **(Postcondition):** An assertion that is guaranteed to hold after c finishes execution, provided P was initially true.

1.1 Partial Correctness

Hoare Logic primarily deals with **Partial Correctness**. A triple $\{P\}\ c\ \{Q\}$ is valid (written $\models \{P\}\ c\ \{Q\}$) if:

> For every initial state σ that satisfies P, if the execution of c starting from σ terminates in a final state σ', then σ' must satisfy Q.

Crucially, partial correctness does not require the program to terminate. If the program loops forever, the triple is vacuously true.

---

2. Inference Rules for Partial Correctness

To prove a triple is valid, we use a set of **Inference Rules** to derive it formally (written $\vdash \{P\}\ c\ \{Q\}$).

2.1 Basic Rules

- **Skip Rule:** The state does not change, so the assertion remains the same.

$$\frac{}{\{P\}\ \texttt{skip}\ \{P\}}$$

- **Assignment Rule:** To ensure P holds after $x := a$, it must hold for the expression a beforehand. $P[a/x]$ denotes the substitution of x with a in P.

$$\frac{}{\{P[a/x]\}\ x := a\ \{P\}}$$

2.2 Structural Rules

- **Sequence Rule:** To reason about $c_1; c_2$, we find an intermediate assertion R.

$$\frac{\{P\}\ c_1\ \{R\} \quad \{R\}\ c_2\ \{Q\}}{\{P\}\ c_1; c_2\ \{Q\}}$$

- **Conditional Rule:** We analyze both branches of an `if` statement.

$$\frac{\{P \wedge b\}\ c_1\ \{Q\} \quad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \texttt{if}\ b\ \texttt{then}\ c_1\ \texttt{else}\ c_2\ \{Q\}}$$

2.3 The While Rule and Invariants

Reasoning about loops requires a **Loop Invariant** P, a property that holds before, during, and after every iteration of the loop.

$$\frac{\{P \wedge b\}\ c\ \{P\}}{\{P\}\ \texttt{while}\ b\ \texttt{do}\ c\ \{P \wedge \neg b\}}$$

- If the invariant P is preserved by the body c when b is true, it will still hold when the loop eventually terminates (at which point b is false).

2.4 Rule of Consequence

This rule allows us to strengthen preconditions or weaken postconditions using standard logic.

$$\frac{P' \implies P \quad \{P\}\ c\ \{Q\} \quad Q \implies Q'}{\{P'\}\ c\ \{Q'\}}$$

---

3. Validity vs. Derivability

There is a distinction between a property being "true" in the model and being "provable" in the logic:

- **Validity ($\models \{P\}\ c\ \{Q\}$):** The property holds according to the denotational/operational semantics.
- **Derivability ($\vdash \{P\}\ c\ \{Q\}$):** The property can be proven using the inference rules listed above.

3.1 Soundness and Completeness

- **Soundness:** Every triple we can derive is actually valid.

$$\vdash \{P\}\ c\ \{Q\} \implies \models \{P\}\ c\ \{Q\}$$

- **Relative Completeness:** Every valid triple can be derived, assuming we have a perfect oracle for the underlying mathematical logic (e.g., arithmetic).

$$\models \{P\}\ c\ \{Q\} \implies \vdash \{P\}\ c\ \{Q\}$$

---

4. Practical Application: Verification Conditions

In practice, human experts or automated tools use **Weakest Preconditions (wp)** to generate **Verification Conditions (VCs)**.

1. Start with the desired postcondition Q.
2. Work backward through the program to find the weakest possible precondition.
3. Check if the given precondition P implies this weakest precondition.

Rule	Backward Step
Assignment	$wp(x := a, Q) = Q[a/x]$
Sequence	$wp(c_1; c_2, Q) = wp(c_1, wp(c_2, Q))$

05. Necessary Condition Logic

This chapter introduces **Necessary Condition Logic (NC Logic)**, which shifts the perspective from forward execution to backward reasoning using **over-approximation**. While Hoare Logic asks "what is guaranteed to happen if we start in P?", NC Logic asks "what *must* have been true at the start if we eventually reached Q?".

---

1. The Core Concept: Backward Over-Approximation

NC Logic is the backward counterpart to Hoare Logic. It is used to find the **necessary conditions** for a specific outcome (e.g., reaching a target state, a bug, or a specific line of code).

1.1 Semantic Definition

We denote an NC triple using angle brackets:

$$\langle P \rangle \; c \; \langle Q \rangle$$

This triple is valid if and only if:

$$[[c]]_{op} Q \subseteq P$$

Meaning: The set of all initial states that can possibly lead to a state in Q via command c ($[[c]]_{op}Q$) is a subset of P. Therefore, starting in P is a *necessary condition* to reach Q. If the initial state was not in P, it is impossible to reach Q.

1.2 Comparison with Hoare Logic

- **Hoare Logic (HL):** $\{P\} \; c \; \{Q\} \iff [[c]]P \subseteq Q$ (Forward Over-approximation).
- **NC Logic:** $\langle P \rangle \; c \; \langle Q \rangle \iff [[c]]_{op}Q \subseteq P$ (Backward Over-approximation).

By expanding the definition, $\langle P \rangle \; c \; \langle Q \rangle$ is equivalent to saying: $\forall \sigma, \sigma'. \; (\sigma, \sigma') \in [[c]] \wedge \sigma' \in Q \implies \sigma \in P$.

---

2. Inference Rules for NC Logic

The inference rules for NC Logic are structurally similar to Hoare Logic but are designed to propagate the post-condition Q backwards to find the necessary pre-condition P.

2.1 Basic Rules

- **Skip:**

$$\frac{}{\langle P \rangle \; \texttt{skip} \; \langle P \rangle}$$

- **Assignment:** The exact set of states that lead to Q after $x := e$ is $Q[e/x]$. Since we are over-approximating, any P that contains $Q[e/x]$ is a valid necessary condition.

$$\frac{}{\langle Q[e/x] \rangle \; x := e \; \langle Q \rangle}$$

2.2 Structural Rules

- **Sequence:**

$$\frac{\langle P \rangle \; c_1 \; \langle R \rangle \quad \langle R \rangle \; c_2 \; \langle Q \rangle}{\langle P \rangle \; c_1; c_2 \; \langle Q \rangle}$$

- **Choice (Non-determinism/If-statements):** If Q can be reached via either branch, the necessary condition must account for both possibilities. Thus, we take the union.

$$\frac{\langle P_1 \rangle\ c_1\ \langle Q \rangle \quad \langle P_2 \rangle\ c_2\ \langle Q \rangle}{\langle P_1 \cup P_2 \rangle\ c_1 + c_2\ \langle Q \rangle}$$

2.3 Rule of Consequence

In NC Logic, since P is an over-approximation of the pre-states, we can safely **weaken the precondition** (make it larger) or **strengthen the postcondition** (make it smaller).

$$\frac{P \implies P' \quad \langle P \rangle\ c\ \langle Q \rangle \quad Q' \implies Q}{\langle P' \rangle\ c\ \langle Q' \rangle}$$

(Note how this is the exact opposite of the Hoare Logic consequence rule).

---

3. Applications

NC Logic is extremely useful in **security analysis and debugging**. If Q represents an exploitation state (e.g., a buffer overflow or unauthorized access), computing $\langle P \rangle\ c\ \langle Q \rangle$ gives us P: the conditions that an attacker *must* satisfy to trigger the exploit. If P evaluates to `false`, the bug is unreachable.

06. Incorrectness Logic

This chapter introduces **Incorrectness Logic (IL)**, a formal system designed for **bug-finding** and proving the **presence of errors** rather than their absence. Unlike Hoare Logic, which uses over-approximation to prove correctness, IL uses **under-approximation** to ensure that every reported bug is a real one (No False Positives).

---

1. Motivation: From Verification to Bug-Finding

While Hoare Logic aims to prove that no bad states are reachable, Incorrectness Logic focuses on proving that a specific bad state *is* reachable.

- **Hoare Logic (Verification):** Focuses on "any accepted program satisfies the property" (Soundness). It is prone to False Positives due to over-approximation.
- **Incorrectness Logic (Bug-Finding):** Focuses on "any reported bug is a real bug". It uses under-approximation to avoid False Positives, though it may have False Negatives (missing some bugs).

---

2. The Incorrectness Triple

The core construct is the **Incorrectness Triple**, which uses square brackets but with a different semantic meaning than Hoare's total correctness:

$$[P]\ c\ [Q]$$

2.1 Semantic Definition

An incorrectness triple $[P]\ c\ [Q]$ is valid if and only if:

$$Q \subseteq [[c]]P$$

This means that **every state** in the post-condition Q is **reachable** from at least one state in the pre-condition P after executing command c.

2.2 Comparison of Triples

- **Hoare (Over-approximation):** $\{P\}\ c\ \{Q\} \iff [[c]]P \subseteq Q$.
- **Incorrectness (Under-approximation):** $[P]\ c\ [Q] \iff Q \subseteq [[c]]P$.

---

3. Inference Rules for Incorrectness Logic

The rules for IL are designed to propagate reachability information forward.

3.1 Basic Rules

- **Assignment Rule:** A state reached after an assignment is reachable if the original state was in P.

$$\frac{}{[P]\ x := e\ [\{\sigma[[[e]]\sigma/x] \mid \sigma \in P\}]}$$

- **Sequence Rule:** Reachability is transitive through sequential execution.

$$\frac{[P]\ c_1\ [R] \quad [R]\ c_2\ [Q]}{[P]\ c_1; c_2\ [Q]}$$

3.2 Choice and Iteration

- **Choice Rule:** Since IL tracks what *can* happen, the choice rule takes the union of reachable states from both branches.

$$\frac{[P]\ c_1\ [Q_1] \quad [P]\ c_2\ [Q_2]}{[P]\ c_1 + c_2\ [Q_1 \cup Q_2]}$$

- **Iteration (Unrolling):** To find bugs in loops, IL typically explores the loop for a finite number of iterations (k).

$$\frac{[P]\ c^k\ [Q]}{[P]\ c^*\ [Q]}$$

  If a state is reachable in exactly k steps, it is reachable in the loop.

---

4. Handling Errors: Real Incorrectness Logic

In real-world analysis, we distinguish between normal termination and error states.

4.1 Exit Conditions

An extended triple tracks the "exit status" ϵ (e.g., `ok` for normal termination, `er` for an error/bug):

$$[P]\ c\ [\epsilon : Q]$$

- **Reachability of Errors:** $[P]\ c\ [\text{er} : Q]$ means every state in Q is an error state reachable from P.
- **Short-circuiting:** If a command c_1 results in an error, the subsequent command c_2 in a sequence $c_1; c_2$ is not executed for that specific trajectory.

4.2 Application to Static Analysis

IL is the foundation for modern "bug-hunting" tools (like **Facebook Infer** or **Meta Zoncolan**). These tools prioritize reporting "True Positives" to avoid "alarm fatigue" among developers.

07. Sufficient Incorrectness Logic

This chapter introduces **Sufficient Incorrectness Logic (SIL)**. While standard Incorrectness Logic (IL) uses forward under-approximation, SIL uses **backward under-approximation**. It is designed to find conditions that are strictly **sufficient** to trigger a specific state or bug.

---

1. The Core Concept: Backward Under-Approximation

SIL is the backward counterpart to Incorrectness Logic. It answers the question: "What starting conditions are *sufficient* to guarantee that a path to Q exists?".

1.1 Semantic Definition

We denote a SIL triple using floor brackets (or sometimes a variant of standard brackets depending on the literature convention):

$$\lfloor P \rfloor\ c\ \lfloor Q \rfloor$$

This triple is valid if and only if:

$$P \subseteq [[c]]_{op} Q$$

Meaning: Every state in P has at least one valid execution path through c that terminates in Q. Therefore, satisfying P is *sufficient* to reach Q.

1.2 Symmetry of the Four Logics

To see the complete picture of Program Analysis logics: 1. **Hoare Logic (HL):** $[[c]]P \subseteq Q$ (Forward Over) - *Absence of bugs.* 2. **Incorrectness Logic (IL):** $Q \subseteq [[c]]P$ (Forward Under) - *Presence of bugs from P.* 3. **Necessary Condition (NC):** $[[c]]_{op} Q \subseteq P$ (Backward Over) - *Required conditions to reach Q.* 4. **Sufficient Incorrectness (SIL):** $P \subseteq [[c]]_{op} Q$ (Backward Under) - *Sufficient conditions to reach Q.*

---

2. Inference Rules for SIL

The rules for SIL mirror those of IL but operate in reverse, propagating the post-condition Q backwards to find a sufficient pre-condition P.

2.1 Basic Rules

- **Skip:**

$$\frac{}{\lfloor P \rfloor \ \texttt{skip} \ \lfloor P \rfloor}$$

- **Assignment:** Since $Q[e/x]$ is the exact set of pre-states, any subset P is a sufficient condition.

$$\frac{}{\lfloor Q[e/x] \rfloor \ x := e \ \lfloor Q \rfloor}$$

2.2 Structural Rules

- **Sequence:**

$$\frac{\lfloor P \rfloor \ c_1 \ \lfloor R \rfloor \quad \lfloor R \rfloor \ c_2 \ \lfloor Q \rfloor}{\lfloor P \rfloor \ c_1; c_2 \ \lfloor Q \rfloor}$$

- **Choice (Non-determinism):** Unlike NC Logic which unions preconditions, in SIL, if P_1 is sufficient to reach Q through c_1, then P_1 is already sufficient for the entire choice construct.

$$\frac{\lfloor P_1 \rfloor \ c_1 \ \lfloor Q \rfloor}{\lfloor P_1 \rfloor \ c_1 + c_2 \ \lfloor Q \rfloor} \quad \text{or} \quad \frac{\lfloor P_2 \rfloor \ c_2 \ \lfloor Q \rfloor}{\lfloor P_2 \rfloor \ c_1 + c_2 \ \lfloor Q \rfloor}$$

2.3 Rule of Consequence

Since SIL tracks an under-approximation (P is a subset of the actual pre-states), we can safely **strengthen the precondition** (make it smaller) or **weaken the postcondition** (make it larger).

$$\frac{P' \implies P \quad \lfloor P \rfloor \ c \ \lfloor Q \rfloor \quad Q \implies Q'}{\lfloor P' \rfloor \ c \ \lfloor Q' \rfloor}$$

---

3. Applications

SIL is the ultimate tool for **automated exploit generation and test-case generation**. If Q is an assertion failure or a crash, finding a valid SIL triple $\lfloor P \rfloor \ c \ \lfloor Q \rfloor$ means that *any* state satisfying P will definitively trigger the bug. You can plug the constraints of P into an SMT solver (like Z3) to generate a concrete test case that is mathematically guaranteed to crash the program.

08. Separation Logic

This chapter introduces **Separation Logic (SL)**, an extension of Hoare Logic designed to facilitate reasoning about programs that mutate shared data structures and pointers. It solves the **aliasing problem** that makes standard Hoare Logic cumbersome when dealing with the heap.

---

1. The Aliasing Problem and Local Reasoning

In standard Hoare Logic, assigning to a pointer (e.g., `*x = 10`) can inadvertently change the value of other variables if they alias the same memory location (e.g., if `x` and `y` point to the same address).

Separation Logic introduces **local reasoning**: specifications and proofs can focus *only* on the portion of memory actually used by a command (its footprint). The **Frame Rule** then allows extending this local proof to a broader, global state without worrying about hidden side-effects.

---

2. Memory Model: Store and Heap

The state in SL is split into two distinct components: * **Store (Stack)** $s : \mathbf{Var} \to \mathbf{Val}$**:** Maps variables to values. * **Heap** $h : \mathbf{Loc} \rightharpoonup \mathbf{Val}$**:** A partial function mapping memory locations (addresses) to values.

Two heaps h_1 and h_2 are **disjoint** (written $h_1 \perp h_2$) if their domains do not overlap ($\mathrm{dom}(h_1) \cap \mathrm{dom}(h_2) = \emptyset$). If they are disjoint, their union $h_1 \uplus h_2$ forms a valid, larger heap.

---

3. Assertions in Separation Logic

SL introduces spatial connectives to express properties about the exact geometry of the heap.

3.1 Spatial Connectives

- **Empty Heap (emp):** Asserts that the heap is currently empty (contains no allocated locations).

$$s, h \models \mathrm{emp} \iff \mathrm{dom}(h) = \emptyset$$

- **Points-to ($E \mapsto E'$):** Asserts that the heap contains *exactly one* allocated location, which is the evaluation of E, and it holds the value E'.

$$s, h \models E \mapsto E' \iff \mathrm{dom}(h) = \{[[E]]s\} \wedge h([[E]]s) = [[E']]s$$

- **Separating Conjunction ($P * Q$):** Asserts that the current heap can be split into two *disjoint* sub-heaps, where one satisfies P and the other satisfies Q.

$$s, h \models P * Q \iff \exists h_1, h_2.\ h_1 \perp h_2 \wedge h = h_1 \uplus h_2 \wedge (s, h_1 \models P) \wedge (s, h_2 \models Q)$$

- **Magic Wand / Separating Implication ($P \mathrel{-\!\!*} Q$):** Asserts that if the current heap is extended with a disjoint heap satisfying P, the combined heap will satisfy Q.

---

4. Inference Rules

SL modifies standard Hoare triples $\{P\}\ c\ \{Q\}$ to enforce a strict **footprint semantics**: c must execute safely (without segmentation faults or accessing unallocated memory) using *only* the memory explicitly described in P.

4.1 The Frame Rule

The cornerstone of local reasoning in SL. It states that if a program c executes safely in a small heap (P), its execution will be identical in a larger heap ($P * R$), leaving the untouched part (R) completely unchanged.

$$\frac{\{P\}\ c\ \{Q\}}{\{P * R\}\ c\ \{Q * R\}}$$

Side condition: The command c must not modify any variables present in the free variables of R.

4.2 Heap Manipulation Rules

These commands map directly to low-level memory operations (dereferencing, mutation, dynamic allocation, and deallocation).

- **Allocation (Cons):** Allocates contiguous memory cells.

$$\frac{}{\{\mathrm{emp}\}\ x := \mathrm{cons}(e_1, \ldots, e_n)\ \{x \mapsto e_1 * \cdots * x + n - 1 \mapsto e_n\}}$$

- **Mutation:** Updates the value at a memory location. The location must be explicitly known to exist in the precondition.

$$\overline{\{E \mapsto -\}\ [E] := E'\ \{E \mapsto E'\}}$$

(Note: $E \mapsto -$ means E points to some arbitrary, unconstrained value).

- **Lookup:** Reads a value from the heap.

$$\overline{\{E \mapsto v\}\ x := [E]\ \{E \mapsto v \wedge x = v\}}$$

(Side condition: x must not appear free in E or v).

- **Deallocation (Dispose):** Frees a memory location. It consumes the points-to assertion, rendering that memory address inaccessible for future operations.

$$\overline{\{E \mapsto -\}\ \text{dispose}(E)\ \{\text{emp}\}}$$

---

5. Memory Safety

By design, Separation Logic triples ensure **memory safety**. If $\{P\}\ c\ \{Q\}$ is provable, it is mathematically guaranteed that c will not cause memory faults (like double-freeing or dereferencing a null/dangling pointer) when executed from a state satisfying P.

6. Inductive Predicates for Data Structures

When dealing with dynamic data structures like linked lists—common in languages with manual memory management—we need a way to describe an arbitrary number of dynamically allocated nodes. Separation Logic handles this elegantly using recursive (inductive) predicates.

6.1 Lists and List Segments

To reason about a list, we define what it means for a pointer to hold a list of a certain shape.

- **List Segment (ls):** A list segment from a pointer x to a pointer y represents a chain of nodes starting at x and ending just before y.

$$ls(x, y) \triangleq (x = y \wedge \text{emp}) \vee (x \neq y \wedge \exists v, l.\ x \mapsto (v, l) * ls(l, y))$$

Base case: x and y are the same pointer, and the heap is empty. *Inductive step:* x points to a node containing a value v and a "next" pointer l, and separately ($*$), there is a list segment from l to y.

- **Null-Terminated List (list):** A standard linked list is simply a list segment that ends at `nil` (or `null`).

$$\text{list}(x) \triangleq ls(x, \text{nil})$$

6.2 Why Standard Conjunction Fails

Using standard Boolean conjunction ($\wedge$) to assert that pointers do not alias requires an $O(n^2)$ explosion of inequalities. For three pointers x, y, z, we would need: $x \neq y \wedge y \neq z \wedge x \neq z$

If we try to define data structures using $\wedge$, we cannot guarantee disjointness. For example, claiming a heap satisfies a list starting at $x \wedge$ a list starting at y does not prevent the two lists from merging or sharing nodes in memory. The separating conjunction ($*$) natively enforces that the memory footprints of the two lists are strictly disjoint: $\text{list}(x) * \text{list}(y)$.

---

7. Practical Reasoning with the Frame Rule

Let's look at how local reasoning applies to practical pointer manipulation, such as traversing or mutating a linked list.

7.1 Example: List Concatenation

Consider the task of appending a list pointed to by y to the end of a non-empty list pointed to by x.

Code snippet:

```
// t traverses the list to find the last node
t := x;
n := [t.next];
while (n != nil) {
    t := n;
    n := [t.next];
}
// append y
[t.next] := y;
```

Proof Sketch:

1. **Precondition:** The lists are completely disjoint. $\{\text{list}(x) * \text{list}(y) \wedge x \neq \text{nil}\}$
2. **Loop Invariant:** As we traverse, we logically split the list x into a processed segment and an unprocessed segment.

   $\{ls(x,t) * t \mapsto (v,n) * \text{list}(n) * \text{list}(y)\}$
3. **Loop Exit:** When the loop terminates, $n = \text{nil}$. The invariant becomes:

   $\{ls(x,t) * t \mapsto (v,\text{nil}) * \text{list}(y)\}$
4. **Mutation:** The assignment `[t.next] := y` updates the "next" field of the last node to point to y. The minimal footprint for this mutation is just $t \mapsto (v, \text{nil})$.

   By the mutation axiom: $\{t \mapsto (v,\text{nil})\}$ $[\texttt{t.next}] := \texttt{y}$ $\{t \mapsto (v,y)\}$
5. **Applying the Frame Rule:** We frame the rest of the lists ($ls(x,t)$ and $\text{list}(y)$) around the local mutation:

   $\{ls(x,t) * t \mapsto (v,\text{nil}) * \text{list}(y)\}$ $[\texttt{t.next}] := \texttt{y}$ $\{ls(x,t) * t \mapsto (v,y) * \text{list}(y)\}$
6. **Postcondition:** The resulting spatial layout $ls(x,t) * t \mapsto (v,y) * ls(y,\text{nil})$ exactly matches the definition of $\text{list}(x)$.

By defining only the exact memory cell being mutated and framing out the rest, Separation Logic avoids the need to globally verify that the mutation did not corrupt other parts of the data structures.

09. Incorrectness Separation Logic

This chapter introduces **Incorrectness Separation Logic (ISL)**, which merges the local reasoning of Separation Logic (SL) with the under-approximation principles of Incorrectness Logic (IL). The goal shifts from proving that a program is perfectly safe to systematically and mathematically proving the presence of memory bugs (like memory leaks, use-after-free, and null-pointer dereferences) without yielding false positives.

---

1. The Need for ISL

Standard Separation Logic is an over-approximation technique: it guarantees the *absence* of memory safety violations. However, when static analysis tools (like Meta's Infer) report a potential issue based on an over-approximation, it might be a false positive, leading to "alarm fatigue" among developers.

Incorrectness Logic (IL) guarantees that any reported bug is a true bug (under-approximation). By combining IL with the spatial reasoning of SL ($*$), we get **ISL**, which allows bug-finding tools to scale to large codebases by analyzing functions locally and framing out the rest of the memory.

---

2. Adapting Separation Logic for Incorrectness

In ISL, we use the incorrectness triple $[P]\ c\ [Q]$, where P and Q now contain spatial assertions.

2.1 The Issue with Deallocation

In standard SL, the `free(x)` (or `dispose(x)`) command consumes the memory cell, leaving an empty heap:

$$\{x \mapsto -\}\ \texttt{free}(\texttt{x})\ \{\text{emp}\}$$

If we attempt to use this directly in an under-approximation setting (ISL), we encounter a problem. In IL, we can drop assertions to widen the precondition or shrink the postcondition. If `free(x)` simply results in `emp`, we lose the explicit information that the memory at x was deallocated. If we later try to dereference x, the logic might not have enough information to decisively prove an error occurred, leading to false negatives (missing real bugs).

2.2 The "Deallocated" Assertion

To solve this, ISL introduces a new spatial assertion to explicitly track freed memory. Instead of simply removing the cell from the domain, we mark it as deallocated (often denoted conceptually as invalid(x) or $x \mapsto$ freed).

$$[x \mapsto v]\ \texttt{free}(\texttt{x})\ [x \mapsto \text{freed}]$$

This is crucial for bug finding: * **Use-After-Free:** If the current state contains $x \mapsto$ freed and the command attempts to read or write to $[x]$, the logic definitively transitions to an error state. * **Double-Free:** If the state contains $x \mapsto$ freed and the command executes `free(x)`, it definitively triggers an error.

---

3. The ISL Frame Rule

The Frame Rule remains the cornerstone for scalability, allowing tools to analyze a small footprint and embed it into a larger context.

$$\frac{[P]\ c\ [\epsilon : Q]}{[P * R]\ c\ [\epsilon : Q * R]}$$

(where ϵ is the exit status: `ok` or `er`, and R is untouched by c)

Compatibility with Freed Memory: The separating conjunction ($*$) works seamlessly with the new deallocated assertion. If we have $(x \mapsto \text{freed}) * (y \mapsto v)$, it guarantees that x and y do not alias. Because x is explicitly marked as freed, y must be a valid, distinct allocated cell.

---

4. Real-World Case Study: Vector Reallocation

A classic bug efficiently found by ISL involves dynamic array reallocations, such as using `std::vector::push_back` in C++.

The Scenario: 1. A program obtains a pointer `p` to an element inside a vector (e.g., `p = &v[0]`). 2. The program calls `v.push_back(new_element)`. 3. The program attempts to read or write using `p`.

The Bug (Use-After-Free): Under the hood, if the vector's capacity is full, `push_back` will allocate a new, larger memory block, copy the existing elements over, and **free the old memory block**. * The pointer `p` still points to the old memory address. * Because the old block was freed, `p` is now a dangling pointer. * Dereferencing `p` at step 3 causes a Use-After-Free error.

ISL can mathematically prove this bug exists: * *Step 1:* State is $p \mapsto v * \text{vector_rest}$. * *Step 2:* The `push_back` semantics explicitly output a state where the old footprint is freed: $p \mapsto \text{freed} * \text{new_vector_allocation}$. * *Step 3:* The read operation strictly requires $p \mapsto -$. Since it encounters $p \mapsto \text{freed}$, the derivation forces a transition to the `er` (error) state.

---

5. Sufficient Incorrectness Separation Logic

Just as standard IL has a backward counterpart, ISL can be run backwards to yield **Sufficient Incorrectness Separation Logic**.

Instead of moving forward to see if a bug is reachable, we start from the error state (e.g., the exact line where `p` is dereferenced while pointing to freed memory) and propagate the constraints *backwards*.

$$\lfloor P \rfloor \; c \; \lfloor \text{er} : Q \rfloor$$

This computes the exact, minimal spatial heap requirements (the Sufficient Condition P) that an attacker or fuzzer needs to provide to guarantee the software crashes. Because it uses local reasoning, the analyzer only tracks the pointers involved in the crash, ignoring the thousands of irrelevant memory cells in the global heap.

10. Abstract Interpretation Foundations

This chapter introduces **Abstract Interpretation**, a universal, mathematically rigorous framework formalized by Patrick and Radhia Cousot (1977). It is used to design static analyses that predict software behavior without executing the code.

Unlike finite-state model checking, Abstract Interpretation easily scales to infinite state spaces by trading exactness for computability. It guarantees **soundness by construction** through systematic over-approximation.

---

1. The Core Idea: Computing on Abstractions

The state space of a real-world program is practically infinite. Tracking the exact set of reachable states (the *Concrete Semantics*) is impossible. Abstract interpretation solves this by mapping the infinite concrete domain into a simpler, mathematical *Abstract Domain* where computations are finite or strictly bounded.

- **Concrete Domain (C):** The exact semantics of the program (e.g., the powerset of all possible memory states, $\wp(\Sigma)$).
- **Abstract Domain (A):** A simplified mathematical structure representing properties of interest (e.g., intervals, signs).
- **Concretization Function ($\gamma : A \to C$):** Maps an abstract element back to the set of concrete states it represents. If the abstract element is "Positive", $\gamma(\text{Positive})$ is the set of all integers > 0.

- **Abstraction Function ($\alpha : C \to A$):** Maps a set of concrete states to the *most precise* (best) abstract element that contains them.

If a static analyzer operates strictly within this framework, the results are mathematically guaranteed to over-approximate the real program behavior, ensuring no bugs of the targeted class are missed (no false negatives).

---

2. Geometric Examples of Abstract Domains

To understand how precision and performance trade off, it helps to visualize abstract domains geometrically, representing program states as points on a 2D plane (e.g., tracking two variables, x and y).

2.1 Non-Relational Domains

These domains abstract each variable independently. They are extremely fast but lack precision when variables interact.

- **Signs Domain:** Tracks whether a variable is $+$, $-$, or 0. Geometrically, this restricts points to specific quadrants of the plane. It is highly imprecise but extremely cheap to compute.
- **Intervals Domain:** Binds variables between a minimum and maximum value: $x \in [min_x, max_x]$ and $y \in [min_y, max_y]$. Geometrically, this draws a bounding box (a rectangle aligned with the axes) around the valid concrete points.

2.2 Relational Domains

In languages like C++, tracking relations between variables is crucial for proving memory safety (e.g., ensuring an index `i` is always strictly less than the array `size`). Relational domains capture these dependencies but at a higher computational cost.

- **Octagons:** Captures relations of the form $\pm x \pm y \leq c$. Geometrically, this creates eight-sided polygons. It accurately tracks bounds like $x \leq y + 5$.
- **Polyhedra:** Captures any linear inequality $\sum a_i x_i \leq c$. Geometrically, this creates arbitrary convex polyhedra. It is extremely precise but computationally expensive (exponential worst-case complexity).

---

3. Compositionality and Abstract Semantics

To automate the analysis, the tool must compute the abstract state step-by-step, mimicking the program's execution but in the abstract domain.

For every concrete operation in the language, we define an **abstract transfer function**.

- **Translation (Assignment):** If the concrete move is $x := x + 1$, the abstract interval operation is $[a, b] \oplus [1, 1] = [a + 1, b + 1]$.
- **Choice (Control Flow):** If the program branches, the abstract analyzer must merge the possible outcomes. It does this using the **Join** operation ($\sqcup$) of the abstract domain. For intervals, the join of $[1, 2]$ and $[5, 6]$ is the bounding box $[1, 6]$. Note the loss of precision: the values 3 and 4 are now included in the over-approximation, even though they are concretely unreachable.

---

4. Loops, Fixpoints, and Widening

Analyzing loops is the most complex part of static analysis. The analyzer must evaluate the loop body repeatedly until the abstract state stabilizes—a process known as computing the **Least Fixed Point (LFP)**.

4.1 The Termination Problem

Consider a simple loop:

```
int x = 0;
while (x < 1000) { x = x + 1; }
```

In the Intervals domain, the iterations would look like this:

1. $[0, 0]$
2. $[0, 0] \sqcup [1, 1] = [0, 1]$
3. $[0, 1] \sqcup [2, 2] = [0, 2]$

The analyzer would have to run 1000 times to reach the stable fixpoint $[0, 1000]$. If the loop condition was `while(true)`, the interval would grow forever, and the analyzer would never terminate.

4.2 The Widening Operator (∇)

To force the analysis to terminate in a finite number of steps, Abstract Interpretation introduces **Widening**.

Widening accelerates convergence by detecting bounds that are changing and aggressively pushing them to $+\infty$ or $-\infty$.

- Instead of standard union ($\sqcup$), we apply widening: $[0, 0] \nabla [0, 1]$.
- The widening operator notices the upper bound changed from 0 to 1. It assumes it will keep growing and instantly approximates the upper bound as $+\infty$.
- The abstract state jumps directly to $[0, +\infty]$.

While widening sacrifices precision (the bound is now artificially high), it guarantees that the static analyzer terminates quickly. A subsequent phase, called **Narrowing** (Δ), can often be applied to recover some of the lost precision by using the loop condition to pull $+\infty$ back down to 1000.

11. Abstract Interpretation Fixpoints and Analysis Design

This chapter dives deeper into the practical application of Abstract Interpretation. We explore the theoretical limits of static analysis, how to compute program semantics using Control Flow Graphs, and how to design abstract transfer functions to reach a fixpoint.

---

1. Decidability and the Limits of Analysis

When analyzing a program, we are interested in its semantic properties (e.g., "does this program terminate?", "does it ever divide by zero?").

According to **Rice's Theorem**, any non-trivial semantic property of a Turing-complete language is **undecidable**. This means it is mathematically impossible to write an algorithm (an analyzer) that can correctly answer "Yes" or "No" for *every* possible program in a finite amount of time.

Because exact analysis is impossible, static analyzers must compromise. They rely on **Safe Approximation**: * **Soundness:** If the analyzer says a program is safe, it *must* be safe. A sound analyzer will never miss a real bug (No False Negatives). * **Imprecision (False Alarms):** To guarantee soundness, the analyzer over-approximates the program's behavior. If an abstract state overlaps with an error state, the analyzer must report a bug, even if that state is concretely unreachable. This results in False Positives.

1.1 Why Trivial Analyzers are Useless

- An analyzer that always outputs "I don't know" or "There might be a bug" is perfectly sound, but completely useless because it lacks **precision**.
- Conversely, a naive syntax-checker (e.g., just looking for the string `/ 0` in the code to detect division by zero) is highly precise but **unsound**, because it will miss hidden bugs (e.g., `x = 0; y = 10 / x;`).

---

2. Control Flow Graphs and Fixpoint Equations

To automate static analysis, we represent the program as a **Control Flow Graph (CFG)**. Each node represents a program point, and edges represent transitions (assignments, guards).

At each node i, we define a set of reachable states S_i. The semantics of the program can be expressed as a system of equations. For example, in a loop: * $S_1 =$ Initial State * $S_2 = S_1 \cup S_{end_of_loop}$ (The loop entry point merges initial states and states returning from the loop body). * $S_3 = f(S_2)$ (Applying the loop body's transfer function to the state).

The analyzer evaluates these equations repeatedly. When the sets stop changing ($S_{new} = S_{old}$), the system has reached a **Fixpoint**. In the concrete domain, this could take infinite time. In an abstract domain, it stabilizes quickly.

---

3. Abstract Semantics in Action: The Factorial Example

Consider a program that computes the factorial of n, storing the result in m:

```
m = 1;
while (n > 0) {
    m = m * n;
    n = n - 1;
}
```

Let's trace this using the **Sign Domain** (Positive, Non-Positive, Zero, $\top$).

1. **Initialization:** m becomes `Positive`. n is unknown ($\top$).
2. **Loop Guard (`n > 0`):** The state splits. Inside the loop, n is strictly `Positive`.
3. **Multiplication (`m = m * n`):** `Positive` * `Positive` = `Positive`. m remains `Positive`.
4. **Subtraction (`n = n - 1`):** `Positive` - 1. Here we lose precision. A positive number minus one could be positive, or it could be zero. In a basic Sign Domain, we must abstract this as `Non-Negative` (or $\top$ if the domain lacks a non-negative element).
5. **Fixpoint:** We feed this back into the loop. Eventually, the analysis concludes that upon exiting the loop, m is `Positive` and n is `Non-Positive`.

3.1 The Problem of Non-Relational Domains

Notice that the analysis successfully proves m is positive, but it cannot prove that $m = n!$. The Sign Domain is **non-relational**; it tracks m and n entirely independently and cannot express mathematical relations between different variables.

---

4. Convergence Strategies: Widening and Refinement

If an abstract domain is infinite (like the Intervals domain), a loop might cause the abstract bounds to grow forever.

4.1 Delayed Widening

Standard **Widening** (∇) detects a growing bound and immediately forces it to infinity ($+\infty$) to ensure the analysis terminates. However, applying it immediately often causes massive precision loss.

- **Strategy:** Perform standard unions ($\sqcup$) for the first 3 or 4 loop iterations to observe the exact behavior and capture precise local bounds. *Then*, apply the widening operator to any bounds that are still changing.

4.2 Domain Refinement

If an analyzer yields too many false alarms, we must **refine the domain** (make it more precise).

- For example, if checking `n > 0` leaves us unable to accurately subtract 1, adding an explicit `Zero` element or upgrading to an `Intervals` domain allows the analyzer to distinguish between exactly 0 and generally ≤ 0.

---

5. Designing Abstract Transfer Functions

To build an abstract interpreter, we must rigorously define how mathematical operations behave on abstract elements. This is usually done via lookup tables.

Consider the **Parity Domain** ($\bot, \text{Even}, \text{Odd}, \top$):

Addition ($\oplus$):

- $\text{Even} \oplus \text{Even} = \text{Even}$
- $\text{Odd} \oplus \text{Even} = \text{Odd}$
- $\text{Odd} \oplus \text{Odd} = \text{Even}$
- $\text{Even} \oplus \top = \top$

Multiplication ($\otimes$):

- $\text{Even} \otimes \text{Even} = \text{Even}$
- $\text{Even} \otimes \text{Odd} = \text{Even}$
- $\text{Odd} \otimes \text{Odd} = \text{Odd}$

Division ($\oslash$): Division introduces extreme precision loss.

- $\text{Even} \oslash \text{Even} = \top$. (e.g., $6/2 = 3$ (Odd), but $8/2 = 4$ (Even). Since it can be either, the sound answer is $\top$).
- $\text{Odd} \oslash \text{Even} = \top$.

If an abstract operation is sound, it must encompass all possible concrete results. If we don't know the exact answer, we fall back to $\top$ (meaning "any value") to maintain soundness.

12. Order Theory and Galois Connections

This chapter formalizes the mathematical foundations of Abstract Interpretation. To rigorously prove that a static analysis is sound and terminates, we must define the structures of our concrete and abstract domains using Order Theory, and formally link them using **Galois Connections**.

---

1. Motivation: Approximating Memory States

Consider analyzing an array access `x[a]`. To prove the access is safe, we must prove that the variable `a` is always within the array bounds (e.g., $0 \leq a \leq 9$).

Tracking the exact concrete memory states (all possible values of `a` and other variables) is often impossible due to state explosion. If we abstract the memory using the **Intervals Domain**, we represent the set of possible values of `a` as a bounding range $[min, max]$.

While this makes the analysis computable, it introduces an inevitable **loss of precision**. For instance, in a non-relational domain like Intervals, we lose the relation between variables (e.g., we might know $a \in [0, 9]$ and $n \in [1, 10]$, but we forget that a strictly depends on n via $a = n - 1$). The formal theory below ensures that despite this loss of precision, the analysis remains strictly **sound** (it never misses a real out-of-bounds error).

---

2. Order Theory Fundamentals

To compute over-approximations systematically, we structure our domains as **Partially Ordered Sets (Posets)**.

2.1 Partial Orders

A partial order $\sqsubseteq$ on a set X is a binary relation that is:

1. **Reflexive:** $\forall x \in X.\ x \sqsubseteq x$
2. **Anti-symmetric:** $\forall x, y \in X.\ (x \sqsubseteq y \wedge y \sqsubseteq x) \implies x = y$
3. **Transitive:** $\forall x, y, z \in X.\ (x \sqsubseteq y \wedge y \sqsubseteq z) \implies x \sqsubseteq z$

In program analysis, the partial order denotes **precision** or **information content**. For example, in the powerset domain $\wp(\mathbb{Z})$, the order is standard set inclusion ($\subseteq$). For the Intervals domain, $[a, b] \sqsubseteq [c, d]$ iff $c \leq a$ and $b \leq d$ (the smaller interval is "more precise").

2.2 Bounds and Complete Lattices

When analyzing control flow (like merging paths after an `if` statement), we need to combine abstract states.

- **Least Upper Bound (LUB / Join / $\sqcup$):** The smallest element that is larger than or equal to all elements in a set. It represents merging information (e.g., $[1, 2] \sqcup [4, 5] = [1, 5]$).
- **Greatest Lower Bound (GLB / Meet / $\sqcap$):** The largest element that is smaller than or equal to all elements in a set. It represents intersecting information.

A poset is a **Complete Lattice** if the LUB ($\sqcup$) and GLB ($\sqcap$) exist for *any* subset of elements, including infinite sets and the empty set.

A complete lattice always has:

- **Bottom ($\bot$):** The LUB of the empty set. It represents unreachable states or the empty set of values.
- **Top ($\top$):** The LUB of the entire domain. It represents "no information" or "any possible value".

2.3 Ascending Chain Condition (ACC)

A poset satisfies the **Ascending Chain Condition (ACC)** if every strictly ascending sequence of elements eventually stabilizes (i.e., $x_0 \sqsubseteq x_1 \sqsubseteq x_2 \ldots$ implies there exists a k such that $x_k = x_{k+1} = \ldots$). If an abstract domain satisfies ACC, fixpoint computations (analyzing loops) are guaranteed to terminate in finite time without needing a Widening operator.

---

3. The Abstraction and Concretization Functions

To relate the Concrete Domain $(C, \subseteq)$ and the Abstract Domain $(A, \sqsubseteq)$, we define two mappings.

3.1 Concretization Function (γ)

The function $\gamma : A \to C$ gives the semantic meaning to an abstract element. It returns the largest set of concrete elements that satisfy the abstract property.

- Example in Signs: $\gamma(\text{Positive}) = \{x \in \mathbb{Z} \mid x > 0\}$.
- Example in Intervals: $\gamma([0, 9]) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

3.2 Abstraction Function (α)

The function $\alpha : C \to A$ takes a set of concrete elements and returns the **most precise** (smallest) abstract element that covers them all.

- Example: $\alpha(\{1, 5, 8\}) = [1, 8]$.

---

4. Galois Connections

A **Galois Connection (GC)** is the formal mathematical framework that ties α and γ together, ensuring that our abstractions are sound.

Let $(C, \subseteq)$ be the concrete domain and $(A, \sqsubseteq)$ be the abstract domain. A pair of functions $\alpha : C \to A$ and $\gamma : A \to C$ forms a Galois Connection, denoted $(C, \subseteq) \overset{\gamma}{\leftrightarrows} (A, \sqsubseteq)$, if and only if for all $c \in C$ and $a \in A$:

$$\alpha(c) \sqsubseteq a \iff c \subseteq \gamma(a)$$

What this means intuitively:

- If we compute an abstraction $\alpha(c)$ and it is tighter than or equal to a, then the exact concrete states c are safely contained within the meaning of a ($\gamma(a)$).
- This biconditional is the ultimate guarantee of **Soundness**. It mathematically enforces that performing operations in the abstract domain will never accidentally exclude a reachable concrete state.

4.1 Four Equivalent Properties

A Galois connection equivalently guarantees these four properties:

1. α is monotonic.
2. γ is monotonic.
3. **Extensivity:** $\forall c \in C.\ c \subseteq \gamma(\alpha(c))$. (If you abstract a concrete set and then concretize it, you get a set that is *at least as large* as the original. You lose precision, but you never lose safety).
4. **Reductivity:** $\forall a \in A.\ \alpha(\gamma(a)) \sqsubseteq a$.

---

5. Galois Insertions

In a standard Galois Connection, the abstract domain might contain redundant elements—multiple different abstract symbols that map to the exact same concrete meaning via γ.

A **Galois Insertion (GI)** is a special, strict case of a Galois connection where the abstract domain contains **no redundancy**.

Formally, a GC is a Galois Insertion if:

$$\alpha(\gamma(a)) = a \quad \text{for all } a \in A$$

Equivalently:

- α is surjective (every abstract element is the abstraction of some concrete set).
- γ is injective (no two different abstract elements have the same concrete meaning).

When building an abstract analyzer, we almost always aim for a Galois Insertion by throwing away "useless" abstract elements, ensuring a one-to-one correspondence between the abstract symbols and the semantic properties they represent.

13. Advanced Domains and Completeness

This chapter explores advanced abstract domains used in static analysis, specifically focusing on the **Congruence Domain** for periodic behaviors and the transition to **Relational Domains**. We also discuss how to combine multiple analyses using **Reduced Products** and introduce the theoretical concept of **Completeness**, which highlights why the way code is written matters just as much as what it computes.

---

1. The Congruence Domain

While the Intervals domain is excellent for finding minimum and maximum bounds, it completely fails to capture "holes" or periodic patterns in data. For example, if a variable `x` is incremented by 4 inside a loop starting from 0, `x` will always be a multiple of 4. Intervals can only say $\texttt{x} \in [0, +\infty]$.

To capture periodic behavior, we use the **Congruence Domain** (also known as the strides domain).

1.1 Representation

An abstract element represents a set of integers defined by a congruence class:

$$a\mathbb{Z} + b \triangleq \{a \cdot k + b \mid k \in \mathbb{Z}\}$$

* a is the multiplier (modulus). * b is the offset (remainder).

Examples: * $2\mathbb{Z} + 0$: Represents all **Even** numbers. * $2\mathbb{Z} + 1$: Represents all **Odd** numbers. * $0\mathbb{Z} + c$: Represents exactly the constant c. * $1\mathbb{Z} + 0$: Represents all integers ($\top$).

1.2 Domain Properties

- **No Infinite Ascending Chains:** A crucial property of the Congruence Domain is that it satisfies the Ascending Chain Condition (ACC). Moving strictly up the lattice implies decreasing the modulus a. Since a is a finite integer, it can only decrease a finite number of times before reaching 1 (which is $\top$). Thus, **fixpoint computations always terminate** without needing a widening operator.

- **Intersection (Meet $\sqcap$):** The meet operation is exact. It computes the intersection of two arithmetic progressions using the Least Common Multiple (LCM) of the moduli.
- **Union (Join $\sqcup$):** The join operation computes a new congruence class that safely over-approximates both inputs, utilizing the Greatest Common Divisor (GCD).

---

2. Relational Domains

Non-relational domains (like Signs, Intervals, and Congruences) abstract the value of each variable independently, mapping $Var \to AbstractValue$. This independence causes massive precision loss when variables are mathematically linked (e.g., $x = y + 1$).

Relational Domains solve this by abstracting the state of multiple variables simultaneously, preserving their correlations.

2.1 Types of Relational Domains

1. **Polyhedra Domain:**
   - Tracks arbitrary linear inequalities: $\sum c_i x_i \leq k$.
   - **Pros:** Extremely precise.
   - **Cons:** Exponential worst-case complexity. Computing unions and fixpoints on arbitrary convex polyhedra is computationally heavy.
2. **Octagon Domain:**
   - Tracks restricted inequalities of the form: $\pm x \pm y \leq k$.
   - **Pros:** A sweet spot between precision and cost. It perfectly captures relations like $x \leq y$ (which is $x - y \leq 0$), crucial for proving array bounds (`index < size`).
   - **Cons:** Cubic time complexity ($O(n^3)$), which is manageable for local analyses.
3. **Difference-Bound Matrices (Zones):**
   - Tracks even simpler relations: $x - y \leq k$.

---

3. Domain Composition: The Reduced Product

Often, a single abstract domain is insufficient. We might want the bounding power of Intervals *and* the parity tracking of Congruences. Instead of inventing a completely new domain from scratch, Abstract Interpretation allows us to compose existing domains.

3.1 Direct Product

The simplest combination is the **Direct Product**. If we have an Interval domain and a Parity domain, the abstract state is just a pair of elements: $([a, b], \text{Parity})$. * The abstract operations are performed pointwise: you update the interval, and you independently update the parity.

3.2 Reduced Product

The Direct Product is suboptimal because the two domains do not share information. The **Reduced Product** improves this by introducing a reduction operator that tightens the abstract state by eliminating impossible combinations.

Example of Reduction: Suppose our analysis yields the direct product state: $([2, 4], \text{Odd})$. * Individually, the interval allows $\{2, 3, 4\}$. * Individually, the parity allows $\{\ldots, -1, 1, 3, 5, \ldots\}$. * The reduction operator intersects their concrete meanings and projects the result back to both domains. The only overlapping value is 3. * The state is "reduced" (tightened) to the much more precise exact state: $([3, 3], \text{Odd})$.

---

4. Completeness in Abstract Interpretation

While **Soundness** guarantees that an abstract analysis will never miss a real behavior (no false negatives), **Completeness** is the dual property: it guarantees that the abstract analysis loses absolutely no precision compared to the concrete semantics.

4.1 Formal Definition

Let f be a concrete operation and $f^{\#}$ be its abstract counterpart. * **Soundness:** $\alpha(f(c)) \sqsubseteq f^{\#}(\alpha(c))$ (The abstract operation is an over-approximation). * **Completeness:** $\alpha(f(c)) = f^{\#}(\alpha(c))$ (The abstract operation is exactly as precise as abstracting the concrete result).

If an operation is complete, it means the abstract domain is perfectly suited to track the effects of that operation without adding any "noise" or false alarms.

4.2 Intentional vs. Extensional Properties

A fascinating result in static analysis is that **completeness is an intentional property, not an extensional one**.

- **Extensional:** Refers to the mathematical function a program computes (its input-output mapping).
- **Intentional:** Refers to *how* the program is written (its specific syntax and control flow).

The Consequence: You can have two programs, P_1 and P_2, that are semantically identical (they compute the exact same mathematical function). However, an abstract analyzer might analyze P_1 with perfect precision (completeness), but fail miserably and lose precision when analyzing P_2.

Because static analysis follows the control flow step-by-step (intentional), a slight syntactic rewrite of a loop or conditional can drastically change whether the analyzer can prove the program safe, even if the runtime behavior is identical. This is why code refactoring is sometimes necessary to make software "verifiable" by static analysis tools.

14. Integrating Abstract Interpretation and Incorrectness Logic

This chapter explores a cutting-edge approach in program analysis: unifying the over-approximation of Abstract Interpretation (AI) with the under-approximation of Incorrectness Logic (IL). The goal is to create a single logical framework capable of definitively proving *both* the absence of bugs (correctness) and the presence of real bugs (incorrectness) without falling victim to false positives or inconclusive results.

---

1. The Dual Perspective: Over vs. Under Approximation

To understand the need for a unified framework, we must contrast the two traditional approaches to static analysis:

- **Over-Approximation (Hoare Logic & Standard AI):** * *Mechanism:* Computes a superset of all reachable states.
  - *Strength:* Proves **Correctness**. If the over-approximated boundary never intersects the "error zone" (the specification violation), the program is mathematically guaranteed to be safe.
  - *Weakness:* **False Positives**. If the over-approximation *does* intersect the error zone, we cannot know if the bug is real or just a phantom artifact of our imprecise abstraction.
- **Under-Approximation (Incorrectness Logic):**
  - *Mechanism:* Computes a strict subset of reachable states.
  - *Strength:* Proves **Incorrectness**. If an error state is found within the under-approximation, it is a guaranteed, reproducible bug (True Positive).
  - *Weakness:* **False Negatives**. If no error is found, we cannot conclude the program is safe, as the analysis might have simply ignored the execution path that leads to the crash.

---

2. A Unified Logical Framework

The new framework combines these two perspectives by enriching the logical triples. Instead of computing just an over-approximation or just an under-approximation, we compute an under-approximation Q whose *abstraction* $\alpha(Q)$ acts as a tight over-approximation.

2.1 The Enriched Triple

We aim for a post-condition Q that satisfies two simultaneous requirements: 1. **Under-Approximation:** $Q \subseteq [[c]]P$ (Every state in Q is genuinely reachable from the precondition P). 2. **Over-Approximating Abstraction:** The abstraction of Q in our chosen domain, $\alpha(Q)$, is large enough to cover the *entire* real semantics: $[[c]]P \subseteq \gamma(\alpha(Q))$.

Why is this powerful? If the "safe area" (the program specification) can be exactly expressed in our abstract domain, we only need to look at our under-approximation Q. * If $\alpha(Q)$ is entirely inside the safe area, the program is **correct**. * If $\alpha(Q)$ falls outside the safe area, and our analysis is *complete*, the bugs indicated are **real**.

---

3. The Completeness Equation

The cornerstone of making this unified logic work is the concept of **Completeness**.

An abstract operation is complete if it loses absolutely no precision compared to executing the concrete semantics and then abstracting the result. Mathematically, the **Completeness Equation** is:

$$\alpha([[c]]P) = [[c]]^{\#}(\alpha(P))$$

If this equation holds, it means our abstract interpreter $[[c]]^{\#}$ is the *best correct approximation* possible. In this scenario: * We don't just have an arbitrary over-approximation; we have the tightest possible bounding box. * If this perfect bounding box hits an error state, we know definitively that the real program also hits that error state.

---

4. Sources of Incompleteness: The Problem with Guards

In practice, achieving perfect completeness is difficult. Operations like assignments ($x := x + 1$) or multiplications are often naturally complete in domains like Intervals or Signs. The primary source of precision loss (incompleteness) comes from **Guards** (conditional tests like `if (x == 0)`).

4.1 The Necessary Condition for Complete Tests

For a conditional test b to be evaluated with perfect completeness in an abstract domain, **both the condition b and its negation $\neg b$ must be exactly expressible** in that domain.

Example in the Interval Domain: Consider the test `x == 0`. * The true branch (`x == 0`) is perfectly expressible as the interval `[0, 0]`. * The false branch (`x != 0`) is **not expressible** as a single interval. It represents two disjoint regions: $[-\infty, -1] \cup [1, +\infty]$. * Because the domain cannot express the "hole" at zero, the abstract analyzer is forced to join them into a single interval $[-\infty, +\infty]$ (or $\top$), losing all precision about the fact that x is specifically *not* zero.

Because $\neg b$ is not expressible, the analysis of the `if/else` construct becomes inherently incomplete.

---

5. Local Completeness

Demanding that an abstract domain be perfectly complete for *all possible inputs* is too strict; almost no non-trivial domain satisfies this.

Instead, the logic relies on **Local Completeness**. We only need the abstract operation to be complete for the *specific precondition* P we are currently analyzing, rather than universally for all $P \in \Sigma$.

- If we know $x \in [-7, 7]$, the test `x > 0` might lose precision.
- But if we know $x \in [1, 7]$ (our specific local precondition), the test `x > 0` is perfectly complete because all possible values already pass the test, requiring no complex splitting or joining in the abstract domain.

By tracking Local Completeness through the derivation tree, the analyzer can verify step-by-step whether it has retained enough precision to guarantee its findings.

---

6. The Consequence Rule in the Unified Logic

Because this logic relies on under-approximation at its base, the Rule of Consequence behaves similarly to Incorrectness Logic, but with a strict boundary condition to preserve the over-approximation guarantee.

$$\frac{P' \implies P \quad [P]\ c\ [Q] \quad Q \implies Q'}{[P']\ c\ [Q']}$$

- **Shrinking the Post-Condition:** We are allowed to "forget" reachable states (moving from a larger Q to a smaller Q').
- **The Boundary Constraint:** We can only drop states *as long as the abstraction remains the same*: $\alpha(Q) = \alpha(Q')$.
  - *Example:* If our abstract domain tracks the Sign of a variable, and our concrete reachable states are $\{1, 3, 100\}$ (Abstraction = `Positive`), we can safely drop 3 and 100 to simplify our proof, leaving just $\{1\}$. The abstraction is still `Positive`.
  - We cannot drop all positive numbers, because that would change the abstraction to $\perp$, violating the bounding guarantee required by the unified framework.

15. Control Flow Analysis (CFA)

This chapter introduces **Control Flow Analysis (CFA)**, a fundamental technique for analyzing programs where the control flow is not explicit and statically determinable from the syntax, as is the case in functional or object-oriented languages.

---

1. The Dispatch Problem in Functional Languages

In traditional imperative languages (like C), building a Control Flow Graph (CFG) is relatively simple: the flow sequentially follows instructions, jumps (`goto`, `if`, `while`), and statically known function calls.

In functional languages (or languages with higher-order functions like JavaScript and Python), functions are **first-class citizens**. They can be passed as arguments, returned as results, and assigned to variables. When the analyzer encounters a function application `x(y)`, it faces the **dispatch problem**: which function will actually be executed at runtime? Since `x` is a variable, its value (and therefore the target of the call) depends on the data flow. Control flow and data flow become inseparable.

To solve this problem, CFA **over-approximates** the set of all possible functions to which `x` could evaluate during execution.

---

2. Syntactic Labeling

Since we cannot rely on an explicit CFG, we work directly on the Abstract Syntax Tree (AST). To accurately track where values are generated and where they flow, we assign a **unique label (l)** to each sub-expression of the program.

Example: Consider the application of the identity function to another identity function: `(fn x => x) (fn y => y)`. With labels, it becomes:

$$[[fn\ x \Rightarrow [x]^1]^2\ [fn\ y \Rightarrow [y]^3]^4]^5$$

Each label represents an exact point in the program where a value can be produced or consumed.

---

3. The 0-CFA Framework

The most basic analysis is **0-CFA** (where "0" indicates that the analysis is *context-insensitive*, meaning it does not distinguish between calls to the same function coming from different contexts). The output of the analysis consists of two mathematical components:

1. **Abstract Cache ($\hat{C}$):** A function that maps each label l (a program point) to a set of abstract values (the functions/closures that can flow through that point).
2. **Abstract Environment ($\hat{\rho}$):** A function that maps each program variable (e.g., x, y) to a set of abstract values (the functions that can be assigned to it).

---

4. Acceptability Relation and Constraints

The analysis does not "compute" the result directly but generates a system of **constraints** based on the code's structure. A solution $(\hat{C}, \hat{\rho})$ is considered **acceptable** if it satisfies all the generated constraints.

The main rules are: * **Function Creation:** If at label l there is a function declaration `fn x => e`, then the abstraction of that function must be contained in the cache of l.

$$\{fn\ x \Rightarrow e\} \subseteq \hat{C}(l)$$

* **Variables:** If there is a variable `x` at label l, everything that can flow into `x` (according to the abstract environment) must also flow into l.

$$\hat{\rho}(x) \subseteq \hat{C}(l)$$

* **Application (The heart of CFA):** For an application $[t_1^{l_1}\ t_2^{l_2}]^l$, for **every** function $(fn\ x \Rightarrow e_0^{l_0})$ that the analysis predicts can flow into $\hat{C}(l_1)$, two constraints must hold: 1. Actual parameters flow into formal parameters: $\hat{C}(l_2) \subseteq \hat{\rho}(x)$ 2. The result of the function body flows into the application result: $\hat{C}(l_0) \subseteq \hat{C}(l)$

By solving this system of constraints until a fixpoint is reached, we obtain a complete map of where every function can end up in the program, thus solving the dispatch problem. "'